Московский государственный университет
имени М. В. Ломоносова

Факультет вычислительной математики и кибернетики
Кафедра исследования операций

Практикум на ЭВМ

Отчет по практическому заданию

студента группы 411

Фостенко Олега Андреевича

Москва, 2026

Содержание

1 Постановка задачи **3**

2 Теоретическое описание метода **4**

2.1 Метод потенциалов (u-v метод) 9

2.2 Транспортная таблица 15

3 Практические аспекты **25**

3.1 Замечания к алгоритму 25

3.2 Программная реализация 25

4 Эксперименты **26**

Заключение **28**

A Приложение **29**

A.1 Программная реализация метода с использованием ЯП Python 29

Список литературы **38**

1 Постановка задачи

Транспортная задача — задача минимизации стоимости грузоперевозок некоторого товара (ресурса) от конечного множества производителей к конечному множеству потребителей.

Пусть есть m производителей (например, складов) и n потребителей (например, заводов). Пусть $a_i \geq 0$ есть количество ресурса в наличии у i-го производителя ($i = 1, \ldots, m$), а $b_j \geq 0$ — количество ресурса, необходимое j-му потребителю ($j = 1, \ldots, n$). Пусть c_{ij} есть стоимость перевозки единицы ресурса от производителя i к потребителю j. Требуется определить количество ресурса (x_{ij}), перевозимое от производителя i к потребителю j, такое, что суммарная стоимость грузоперевозок минимальна. Формально:

$$f(x) = \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} x_{ij} \to \min,\ x \in \mathbb{R}^{m \times n} \tag{1}$$

$$\sum_{j=1}^{n} x_{ij} = a_i,\ i = 1, \ldots, m \tag{2}$$

$$\sum_{i=1}^{m} x_{ij} = b_j,\ j = 1, \ldots, n \tag{3}$$

$$x_{ij} \geq 0,\ i = 1, \ldots, m,\ j = 1, \ldots, n \tag{4}$$

Условие (4) накладывает естественные ограничения неотрицательности на количество перевозимого товара. Условие (2) задает величину предложения со стороны каждого производителя, условие (3) задает величину спроса со стороны каждого потребителя.

Нетрудно заметить, что данная задача есть частный случай задачи линейного программирования. Действительно, можно считать, что x, c являются векторами из пространства $\mathbb{R}^{mn}$. Тогда получаем классическую задачу линейного программирования относительно mn переменных при $m + n$ ограничениях-равенствах и mn ограничениях-неравенствах, задающих неотрицательность переменных.

Ограничения (2), (3) представляются в наглядной форме, раскрывающей специальную структуру данной задачи:

$$
\begin{array}{rl}
x_{11} + x_{12} + \cdots + x_{1n} & = a_1 \\
x_{21} + x_{22} + \cdots + x_{2n} & = a_2 \\
\vdots & \vdots \\
x_{m1} + x_{m2} + \cdots + x_{mn} & = a_m
\end{array}
\tag{$\hat{2}$}
$$

$$
\begin{array}{rl}
x_{11} + x_{21} + \ldots + x_{m1} & = b_1 \\
x_{12} + x_{22} + \ldots + x_{m2} & = b_2 \\
\vdots \quad \vdots \quad \vdots \quad \vdots & \vdots \\
x_{1n} + x_{2n} + \ldots + x_{mn} & = b_n
\end{array}
\tag{$\hat{3}$}
$$

2 Теоретическое описание метода

В дальнейшем полагается, что читатель ознакомлен с основной терминологией, связанной с задачами линейного программирования [4; 5]. Термины “базисное допустимое решение” и “вершина” будут использоваться взаимозаменяемо, под “допустимым решением” будет пониматься элемент допустимого множества. Под “решением” или “оптимальным решением” задачи ЛП будет пониматься ее глобальное решение.

Ограничения задачи ЛП задают полиэдр. Известно, что если допустимое множество ограничено и непусто (т.е. является непустым ограниченным полиэдром), то задача ЛП имеет решение и у этого полиэдра есть хотя бы одна вершина. Также известно, что если задача ЛП имеет решение и у полиэдра, задаваемого ограничениями, есть хотя бы одна вершина, то существует вершина этого полиэдра, являющаяся (глобальным) решением этой задачи ЛП.

Из (2), (3) следует утверждение.

Утверждение 1 (Существование допустимого решения)**.** Транспортная задача (1)–(4) имеет допустимое решение тогда и только тогда, когда выполняется равенство

$$
\sum_{i=1}^{m} a_i = \sum_{j=1}^{n} b_j. \tag{5}
$$

Доказательство. Необходимость: если есть допустимое решение x^*, тогда для него

выполнено (2) и (3). Суммируя, получаем:

$$\sum_{i=1}^{m} a_i = \sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij}^* = \sum_{j=1}^{n}\sum_{i=1}^{m} x_{ij}^* = \sum_{j=1}^{n} b_j$$

Достаточность: пусть выполнено равенство (5). Тогда существование допустимого решения следует из процедуры нахождения начального допустимого решения (см. далее). □

Следствие 1 (Существование оптимального решения)**.** При выполнении условия (5) решение задачи (1)–(4) существует и является вершиной полиэдра, задаваемого ограничениями (2)–(4) этой задачи.

Доказательство. Полиэдр, задаваемый (2)–(4), ограничен. Действительно, нетрудно заметить, что каждая из компонент x_{ij} элементов x из допустимого множества удовлетворяет:

$$x_{ij} \leq \max\{\max_{1\leq i\leq m} a_i,\ \max_{1\leq j\leq n} b_j\}$$

Условие (5) гарантирует, что допустимое множество непусто. Получаем, что допустимое множество есть непустой ограниченный полиэдр. Отсюда, задача ЛП имеет решение, причем решением является в том числе хотя бы одна из вершин полиэдра, задаваемого ограничениями. □

Алгоритм нахождения решения задачи (1)–(4) основан на переборе вершин полиэдра, задаваемого ограничениями, что обосновывается доказанными утверждениями.

Рассмотрим процедуру нахождения начального допустимого решения[3] (условие (5) полагается выполненным).

Алгоритм 1 (метод северо-западного угла)

1. Пусть изначально весь товар лежит на складах. Распределим его по потребителям.

Пусть $\alpha_i := a_i,\ i = 1, \ldots, m$ (текущий неиспользованный товар),

$\beta_j := b_j,\ j = 1, \ldots, n$ (текущий неудовлетворенный спрос).

Присвоим также $x_{ij} := 0,\ i = 1, \ldots, m,\ j = 1, \ldots, n$ (т.е. изначально товар никуда не доставлен).

Положим $i := 1,\ j := 1$ (начнем распределять товар, начиная с 1-го производителя 1-му потребителю).

2. Присваиваем $x_{ij} := min\{\alpha_i, \beta_j\}$.

Либо производитель i может доставить весь свой товар потребителю j ($\alpha_i \geq \beta_j$), либо спрос со стороны потребителя j превосходит возможности производителя i ($\alpha_i < \beta_j$).

Если ($\alpha_i \geq \beta_j$), перейти на шаг 4, иначе перейти на следующий шаг.

3. (случай $x_{ij} = \alpha_i < \beta_j$)

Присваиваем $\beta_j := \beta_j - x_{ij}$.

Присвоить $i = i + 1$ (далее спрос потребителя j будет удовлетворять производитель со следующим номером).

Перейти на шаг 5.

4. (случай $x_{ij} = \beta_j \leq \alpha_i$)

Присваиваем $\alpha_i := \alpha_i - x_{ij}$

Присвоить $j = j + 1$ (далее производитель i будет удовлетворять спрос следующего потребителя).

5. (проверка условия останова)

Если $j > n$, завершить, выдав x в качестве ответа алгоритма (x — начальное базисное решение).

Иначе повторить с шага 2.

Утверждение 2 (Ранг системы ограничений)**.** Ранг системы (2)–(3) равен $m + n - 1$.

Доказательство. Из формы ($\hat{2}$), ($\hat{3}$) видно, что сумма строк матрицы ограничений, отвечающий ограничениям ($\hat{2}$), равна сумме строк, отвечающих ограничениям ($\hat{3}$). Отсюда, $rank \leq m + n - 1$.

Обозначим R_i, $i = 1, \ldots, m$ строки, отвечающие соответственно ограничениям ($\hat{2}$) системы ($\hat{2}$)–($\hat{3}$), а C_j, $j = 1, \ldots, n$ — строки, отвечающие ограничениям ($\hat{3}$).

Покажем линейную независимость $m + n - 1$ строк системы. Выберем m строк $R_1, \ldots, R_m$ и первые $n-1$ строк $C_1, \ldots, C_{n-1}$. Предположим существует ненулевая линейная комбинация

$$\sum_{i=1}^{m} \alpha_i R_i + \sum_{j=1}^{n-1} \beta_j C_j = 0.$$

Рассмотрим переменные x_{in}, $i = 1, \ldots, m$. Эти переменные встречаются только в соответствующих R_i и не входят ни в один из выбранных C_j. Следовательно, коэффициенты $\alpha_i = 0$, $i = 1, \ldots, m$.

Так, исходная линейная комбинация представима в виде:

$$\sum_{j=1}^{n-1} \beta_j C_j = 0.$$

Строки C_j, $j = 1, \ldots, n$, очевидно, линейно независимы. Действительно, рассмотрим переменные x_{ij_0} для некоторого j_0. Эти переменные встречаются только в строке C_{j_0} среди выбранных строк. Следовательно, $\sum_j \beta_j C_j = 0 \implies \beta_j = 0$, $j = 1, \ldots, n$

Таким образом, выбранные $m + n - 1$ строк независимы. Отсюда, $rank \geq m + n - 1$, и по доказанному ранее $rank = m + n - 1$.

Заметим также, что любые $m+n-1$ строк системы (2)–(3) линейно независимы, так как аналогичные рассуждения полностью справедливы для выбора любых различных $m+n-1$ строк. □

Далее для сокращения записи используем обозначение ij для номера переменной, понимая под этим упорядоченную пару значений (i, j).

Следствие 2 (Характеризация базисных допустимых решений)**.** В любом базисном допустимом решении не более $m + n - 1$ ненулевых переменных x_{ij}.

Доказательство. Рассмотрим базисное допустимое решение x^*. Пусть ненулевые переменные имеют номера из множества $J(x^*) = \{i_1j_1, i_2j_2, \ldots, i_sj_s\}$ (т.е. $x^*_{ij} > 0 \iff ij \in J(x^*)$). Допустимое решение есть вершина канонического полиэдра, задаваемого ограничениями системы (2)–(4), тогда и только тогда, когда столбцы системы (2)–(3), отвечающие переменным с номерами из множества $J(x^*)$, являются линейно независимыми. По доказанному утверждению, в любом множестве линейно независимых столбцов их не более $m + n - 1$, откуда вытекает требуемое. Действительно, если предположить, что в вершине более $m + n - 1$ переменных положительны, то линейная независимость столбцов, отвечающих им, невозможна. □

Утверждение 3 (Начальное допустимое решение)**.** Допустимое решение, полученное алгоритмом 1, является базисным.

Доказательство. Заметим, что процедура, описанная в алгоритме 1 присваивает ненулевые значения не более чем $m + n - 1$ переменным, что удовлетворяет следствию 2.

Столбец v_{ij} матрицы ограничений (2)–(3), отвечающий переменной x_{ij}, представляется в виде вектора из $\mathbb{R}^{m+n}$ в следующем виде:

$$v_{ij}[k] = \begin{cases} 1, & k = i \\ 1, & k = m + j \\ 0, & \text{иначе} \end{cases}$$

Здесь $v[k]$ обозначает номер компоненты вектора.

Снова обозначим $J(x^*) = \{ij : x^*_{ij} > 0\}$, x^* — допустимое решение, полученное алгоритмом 1. Покажем, что множество столбцов $\{v_{ij} : ij \in J(x^*)\}$ линейно независимо, т.е.:

$$\sum_{ij \in J(x^*)} \lambda_{ij} v_{ij} = 0 \implies \lambda_{ij} = 0 \ \forall ij \in J(x^*).$$

Алгоритм 1 присваивает возможно ненулевые значения переменным в порядке, начиная с x_{11}, на каждом шаге обновляя либо номер строки: $i := i+1$, либо номер столбца: $j := j+1$

Проведем доказательство по индукции:

База индукции: Столбец, отвечающий x_{11}, как и любой другой столбец соответствующей системы, содержит две ненулевые компоненты, а значит образует линейно независимую систему (из одного столбца).

Шаг индукции: Пусть первые $k - 1$ переменных соответствуют линейно независимым столбцам. Рассмотрим k-ю присвоенную переменную x_{ij}.

В случае, если предыдущая присвоенная переменная была $x_{i-1,j}$, номер i пары номеров ij, соответствующих данной переменной, встречается впервые среди всех ранее присвоенных переменных. Отсюда, для всех столбцов имеющейся системы из $k-1$ линейно независимых столбцов i-й элемент этих столбцов есть 0, тогда как для текущего столбца он есть 1. Значит, столбец, соответствующий текущей переменной, линейно независим со всеми остальными в системе, и новая система из k столбцов также линейно независима.

В случае, если предыдущая присвоенная переменная была $x_{i,j-1}$, рассуждения аналогичны, только теперь в столбце, отвечающем переменной x_{ij}, впервые встречается ненулевой элемент с номером $m + j$, откуда и в данном случае система из k столбцов,

получающаяся добавлением текущего столбца к имеющейся системе из $k-1$ столбцов, также будет линейно независима.

Если для допустимого решения x^* столбцы системы неравенств, отвечающие переменным $x^*_{ij} : ij \in J(x^*)$, линейно независимы, то x^* — вершина, то есть базисное допустимое решение. Утверждение доказано. $\square$

2.1 Метод потенциалов (u-v метод)

Метод потенциалов[2] используется для проверки оптимальности базисного допустимого решения транспортной задачи и выбора нового базисного допустимого решения с целью минимизации суммарной стоимости перевозок.

Пусть x^* — базисное допустимое решение транспортной задачи. Введем двойственные переменные (потенциалы):

$$u_i,\ i = 1, \ldots, m \quad \text{для производителей}, \quad v_j,\ j = 1, \ldots, n \quad \text{для потребителей}.$$

Эти переменные выбираются так, чтобы выполнялось условие потенциальности для всех базисных переменных:

$$x^*_{ij} \text{— базисная} \implies c_{ij} = u_i + v_j. \tag{6}$$

Далее базисными переменными в точке x в случае наличия $m+n-1$ ненулевой переменной в этой точке будем называть $x_{ij} : x_{ij} > 0$. Остальные переменные $x_{ij} : x_{ij} = 0$ соответственно назовем небазисными.

Отметим, что в вершине может быть строго меньше $m+n-1$ ненулевых переменных. Эта ситуация отвечает вырожденной вершине. Тогда можно ко множеству переменных $x_{ij} > 0$ добавить нулевые переменные так, чтобы всего переменных в наборе стало $m+n-1$, а столбцы матрицы ограничений-равенств, соответствующие всем переменным набора, образовывали линейно независимую систему (ранг системы равен $m+n-1$, значит это можно сделать).

При применении алгоритма 1 базисными будут все вершины, пройденные в процессе выполнения, даже если они оказались нулевыми. Также возможно, что алгоритм завершится при $i < m$. Эта ситуация возникает при наличии нулевых a_i. В таком случае после завершения алгоритма добавляем к базисным переменные $x_{i+1,n}, \ldots, x_{m,n}$. Нетрудно проверить, что такая процедура породит в итоге в точности $m+n-1$ базисных переменных.

Итак, имеем набор из $m+n-1$ базисных переменных.

Утверждение 4 (Нахождение потенциалов)**.** Потенциалы, отвечающие базисному допустимому решению, удовлетворяющие (6), всегда можно найти.

Доказательство. Для всех $m+n-1$ базисных переменных $ij \in B$ запишем систему линейных уравнений относительно потенциалов:

$$u_i + v_j = c_{ij} \ \forall ij \in B. \tag{7}$$

Система содержит $m+n-1$ уравнений для $m+n$ неизвестных $(u_1, \ldots, u_m, v_1, \ldots, v_n)$. Обозначим матрицу коэффициентов этой системы через $M \in \mathbb{R}^{(m+n-1)\times(m+n)}$. Каждое уравнение соответствует строке матрицы M с единицами в позициях i и $m+j$ и нулями в остальных позициях.

Для базисного допустимого решения столбцы с номерами из B линейно независимы. Как нетрудно заметить, строка системы (7), отвечающая номеру ij, в точности есть транспонированный столбец матрицы системы (2)–(3) с номером ij. Следовательно, для любого базисного допустимого решения система (7) имеет ранг $m+n-1$. Действительно, в системе $m+n-1$ уравнение при линейной независимости всех строк данной системы.

Так как число неизвестных равно $m+n > m+n-1$, система является недоопределенной, а значит, существует хотя бы одно решение. Для однозначности можно зафиксировать один потенциал, например $u_1 = 0$, после чего решение становится единственным.

Следовательно, потенциалы $(u_1, \ldots, u_m, v_1, \ldots, v_n)$ всегда существуют. □

Для всех небазисных переменных вычисляется величина

$$\Delta_{ij} := c_{ij} - (u_i + v_j). \tag{8}$$

Утверждение 5 (Критерий оптимальности по потенциалам)**.** Пусть x^* — базисное допустимое решение транспортной задачи (1)–(4). Пусть потенциалы $u_i,\ i = 1, \ldots, m$, и $v_j,\ j = 1, \ldots, n$ выбраны согласно (6).

Тогда если для всех небазисных переменных выполнено неравенство

$$\Delta_{ij} := c_{ij} - (u_i + v_j) \geq 0,$$

то решение x^* является оптимальным.

Доказательство. Рассмотрим произвольное допустимое решение $x = (x_{ij})$. Запишем значение целевой функции в виде

$$\sum_{i=1}^{m}\sum_{j=1}^{n} c_{ij}x_{ij} = \sum_{i,j}(u_i + v_j)x_{ij} + \sum_{i,j}\big(c_{ij} - (u_i + v_j)\big)x_{ij}.$$

Рассмотрим первое слагаемое:

$$\sum_{i,j}(u_i + v_j)x_{ij} = \sum_{i=1}^{m} u_i \sum_{j=1}^{n} x_{ij} + \sum_{j=1}^{n} v_j \sum_{i=1}^{m} x_{ij}.$$

С учетом ограничений (2)–(3) получаем:

$$\sum_{i,j}(u_i + v_j)x_{ij} = \sum_{i=1}^{m} u_i a_i + \sum_{j=1}^{n} v_j b_j.$$

Рассмотрим второе слагаемое. Разделим сумму по базисным и небазисным переменным:

$$\sum_{i,j}\big(c_{ij} - (u_i + v_j)\big)x_{ij} = \sum_{ij\in B(x^*)} \Delta_{ij}x_{ij} + \sum_{ij\in N(x^*)} \Delta_{ij}x_{ij},$$

где $B(x^*)$ — множество номеров базисных переменных в x^*, а $N(x^*)$ — множество номеров небазисных переменных в x^*.

По определению потенциалов для всех $ij \in B(x^*)$ выполнено $\Delta_{ij} = 0$. По предположению утверждения $\Delta_{ij} \geq 0$ для всех $ij \in N(x^*)$, $x_{ij} \geq 0$ для любого допустимого решения. Следовательно,

$$\sum_{i,j}\big(c_{ij} - (u_i + v_j)\big)x_{ij} \geq 0.$$

Таким образом, для любого допустимого решения x справедливо неравенство

$$\sum_{i,j} c_{ij}x_{ij} \geq \sum_{i=1}^{m} u_i a_i + \sum_{j=1}^{n} v_j b_j.$$

С другой стороны, для решения x^* имеем

$$\sum_{i,j} c_{ij}x^*_{ij} = \sum_{i=1}^{m} u_i a_i + \sum_{j=1}^{n} v_j b_j,$$

поскольку все небазисные переменные в точке x^* равны нулю.

Следовательно, значение целевой функции в точке x^* не превосходит ее значения ни в одной другой допустимой точке, то есть x^* является оптимальным решением транспортной задачи. □

Утверждение 6 (Улучшение неоптимального решения)**.** Пусть x^* — невырожденное базисное допустимое решение транспортной задачи (1)–(4), пусть u_i, v_j — потенциалы для базисных переменных, заданные согласно 6, и пусть существует небазисная переменная x_{kl}, для которой

$$\Delta_{kl} := c_{kl} - (u_k + v_l) < 0.$$

Тогда можно построить новое допустимое решение x^{**}, такое что

$$\sum_{i,j} c_{ij} x_{ij}^{**} < \sum_{i,j} c_{ij} x_{ij}^{*}.$$

Иными словами, решение x^* не является оптимальным.

Доказательство. Пусть x^* — невырожденное базисное допустимое решение с $m+n-1$ базисными переменными, и пусть x_{kl} — небазисная переменная с $\Delta_{kl} < 0$.

Рассмотрим систему ограничений задачи:

$$\sum_{j=1}^{n} x_{ij} = a_i, \quad i = 1, \ldots, m, \qquad \sum_{i=1}^{m} x_{ij} = b_j, \quad j = 1, \ldots, n,$$

которая может быть записана в виде $Ax = b$, где $x \in \mathbb{R}^{mn}$, а A — матрица коэффициентов.

Так как x^* — базисное решение, столбцы матрицы A, соответствующие базисным переменным, линейно независимы. Добавление небазисной переменной x_{kl} к базису делает соответствующий набор столбцов линейно зависимым. Следовательно, существует ненулевой вектор направления $d \in \mathbb{R}^{mn}$, такой что

$$Ad = 0, \quad d_{kl} = 1, \tag{9}$$

и $d_{ij} = 0$ для небазисных переменных, кроме x_{kl}. Действительно, т.к. ранг системы ограничений-равенств есть $m+n-1$, то новый столбец, добавленный к системе из $m+n-1$ столбцов, не может быть линейно независим с остальными. Значит, добавленный столбец выразим через остальные, что и выражает (9).

Введем параметр $\theta > 0$ и рассмотрим семейство решений

$$x(\theta) := x^* + \theta d.$$

Так как $Ad = 0$, выполняются ограничения по суммам строк и столбцов: $Ax(\theta) = b$.

Существуют значения $\theta \in (0, \theta_{\max}]$, при которых $x(\theta) \geq 0$, то есть решение остается допустимым, причем из невырожденности вершины следует положительность этих значений.

Теперь рассмотрим изменение целевой функции вдоль направления d:

$$f(x(\theta)) - f(x^*) = \sum_{i,j} c_{ij}(x_{ij}(\theta) - x^*_{ij}) = \sum_{i,j} c_{ij} d_{ij}\,\theta.$$

c_{ij} выражается в виде $c_{ij} = u_i + v_j + \Delta_{ij}$. Для базисных переменных $\Delta_{ij} = 0$, а для небазисной переменной $\Delta_{kl} < 0$.

Тогда

$$\sum_{i,j} c_{ij} d_{ij} = \sum_{i,j} (u_i + v_j) d_{ij} + \sum_{i,j} \Delta_{ij} d_{ij}.$$

Рассмотрим

$$\sum_{i,j} (u_i + v_j) d_{ij} = \sum_{i,j} u_i d_{ij} + \sum_{i,j} v_j d_{ij}.$$

Разобьем каждую сумму по строкам и столбцам:

$$\sum_{i,j} u_i d_{ij} = \sum_{i=1}^{m} u_i \sum_{j=1}^{n} d_{ij}, \qquad \sum_{i,j} v_j d_{ij} = \sum_{j=1}^{n} v_j \sum_{i=1}^{m} d_{ij}.$$

Так как d — допустимое направление, выполняется $Ad = 0$, то есть

$$\sum_{j=1}^{n} d_{ij} = 0 \quad \forall i, \qquad \sum_{i=1}^{m} d_{ij} = 0 \quad \forall j.$$

Следовательно,

$$\sum_{i,j} u_i d_{ij} = \sum_{i=1}^{m} u_i \cdot 0 = 0, \qquad \sum_{i,j} v_j d_{ij} = \sum_{j=1}^{n} v_j \cdot 0 = 0.$$

Отсюда

$$\sum_{i,j} (u_i + v_j) d_{ij} = 0.$$

Так, лишь вводимая небазисная переменная дает ненулевой вклад в сумму:

$$\sum_{i,j} c_{ij} d_{ij} = \sum_{i,j} (u_i + v_j) d_{ij} + \sum_{i,j} \Delta_{ij} d_{ij} = \sum_{i,j} \Delta_{ij} d_{ij} = \sum_{i,j: d_{ij} \neq 0} \Delta_{ij} d_{ij}.$$

$$\sum_{i,j} \Delta_{ij} d_{ij} = \Delta_{kl} \cdot d_{kl} = \Delta_{kl} < 0.$$

Таким образом,

$$f(x(\theta)) - f(x^*) = \Delta_{kl}\,\theta < 0,$$

и целевая функция уменьшается вдоль направления d, то есть x^* неоптимально. $\square$

Следствие 3 (Выбор нового базисного допустимого решения)**.** При следовании процедуре построения нового допустимого решения в утверждении 6 можно выбрать параметр θ так, что получившееся допустимое решение будет базисным.

Доказательство. Пусть x^* — базисное допустимое решение с множеством базисных переменных B, $|B| = m + n - 1$, и пусть x_{kl} — небазисная переменная с $\Delta_{kl} < 0$.

Рассмотрим снова направление $d \in \mathbb{R}^{mn}$, такое что

$$Ad = 0, \quad d_{kl} = 1.$$

Новое решение, улучшающее целевую функцию:

$$x(\theta) = x^* + \theta d.$$

Допустимое множество ограничено, значит хотя бы одна компонента d является отрицательной.

Выберем $\theta_{\max} > 0$ как

$$\theta_{\max} = \min_{ij: d_{ij} < 0} \frac{x_{ij}}{|d_{ij}|} \tag{10}$$

Хотя бы одна базисная переменная x_{pq} станет равной нулю в точке $x(\theta) = x^* + \theta_{\max} d$. Выберем одну из таких переменных (переменная “выходит из базиса”), а x_{kl} становится положительной (переменная “входит в базис”). Таким образом, новое множество базисных переменных.

$$B' = B \cup \{kl\} \setminus \{pq\}$$

содержит ровно $m + n - 1$ элементов.

Остается проверить линейную независимость столбцов системы A для нового базиса. Поскольку $Ad = 0$, имеем

$$A_{kl} + \sum_{ij \in B} d_{ij} A_{ij} = 0,$$

и $d_{pq} \neq 0$, $pq \in B$ (покидает базис). При этом известно, что A_{kl} выразим через столбцы с номерами B, а они линейно независимы, значит выразим единственным образом. При этом коэффициент $d_{pq} \neq 0$. Стало быть, A_{kl} невыразим через столбцы с номерами B (а они линейно независимы). Отсюда, вся система столбцов с номерами B' линейно независима.

Так, выбрав параметр θ определенным образом, получили новое базисное допустимое решение, улучшающее целевой функционал. $\square$

Замечание: в случае наличия вырожденных вершин используются методы для избежания зацикливания. Например, рандомизация или правило Бленда[1].

2.2 Транспортная таблица

Для практического применения алгоритмов решения транспортной задачи (в частности, метода северо-западного угла и метода потенциалов) удобно использовать специальное представление, называемое транспортной таблицей[2].

Транспортная таблица представляет собой прямоугольную таблицу размера $(m+1) \times (n+1)$ и строится согласно алгоритму, описанному далее.

Каждая строка i таблицы соответствует одному производителю ($i = 1, \ldots, m$). Каждый столбец j таблицы соответствует одному потребителю ($j = 1, \ldots, n$). Ячейка таблицы на пересечении строки i и столбца j соответствует переменной x_{ij} и стоимости перевозки c_{ij}. Последний столбец содержит величины запасов производителей a_i и потенциалы u_i. Последняя строка содержит величины спроса потребителей b_j и потенциалы v_j.

Каждая внутренняя ячейка таблицы соответствует паре индексов (i, j) и содержит две величины: стоимость перевозки c_{ij} и величину перевозимого груза x_{ij}.

На первом этапе в таблицу заносятся только стоимости c_{ij}, а все значения x_{ij} полагаются равными нулю. Также в последнем столбце и последней строке фиксируются величины a_i и b_j.

Далее применяется метод северо-западного угла (алгоритм 1). В результате получается допустимое базисное решение, содержащее не более $m + n - 1$ ненулевых переменных x_{ij}. Согласно этому же алгоритму выделяются базисные переменные (описанным ранее образом добавляются нулевые, если вершина вырождена).

Для применения метода потенциалов к транспортной таблице дополнительно вводятся потенциалы u_i для строк (производителей) и потенциалы v_j для столбцов (потребителей).

Потенциалы определяются из условий

$$u_i + v_j = c_{ij}$$

для всех базисных ячеек таблицы.

Для каждой небазисной ячейки (i, j) вычисляется величина

$$\Delta_{ij} = c_{ij} - (u_i + v_j).$$

Значения Δ_{ij} обычно выписываются в соответствующих ячейках или рядом с ними.

Если для всех небазисных ячеек $\Delta_{ij} \geq 0$, то текущее решение является оптимальным. Если существует ячейка с $\Delta_{ij} < 0$, то она выбирается для улучшения решения.

Для выбранной небазисной ячейки с отрицательной оценкой в транспортной таблице строится замкнутый цикл, проходящий только через базисные ячейки и данную небазисную ячейку, причем передвижения происходят только по строкам и столбцам, в цикле чередуются знаки "+" и "−", в начальной небазисной ячейке ставится знак "+".

По минимальному значению среди ячеек со знаком "−" определяется шаг θ, после чего значения x_{ij} обновляются.

После изменения значений x_{ij} одна из базисных переменных становится равной нулю и выходит из базиса, небазисная переменная входит в базис, пересчитываются потенциалы и оценки Δ_{ij}.

Процедура повторяется до выполнения условия оптимальности.

До сих пор были обоснованы все шаги описанного алгоритма, кроме специфичного для данной задачи способа определения направления d убывания целевой функции.

Следствие 4 (Построение направления убывания)**.** Пусть выполнены условия утверждения 6 за исключением, быть может, невырожденности базисного допустимого решения. Тогда направление убывания/невозрастания функции, получаемое способом, описанным в утверждении 6, имеет следующие свойства:

1) все ненулевые компоненты d_{ij} соответствуют переменным, образующим замкнутый цикл через базисные переменные и переменную, вводимую в базис (x_{kl});

2) значения $d_{ij} \in \{-1, +1\}$ и чередуются по циклу, начиная с $d_{kl} = +1$;

Доказательство. Пусть x^* — рассматриваемое базисное допустимое решение с множеством номеров базисных переменных B, N — множество номеров небазисных переменных, кроме x_{kl}. $|B| = m + n - 1$, и пусть x_{kl} — небазисная переменная, выбранная для введения в базис. Рассмотрим направление $d \in \mathbb{R}^{mn}$, удовлетворяющее

$$Ad = 0, \quad d_{kl} = 1, \quad d_{ij} = 0 \ \ \forall ij \in N.$$

$Ad = 0$ эквивалентно:

$$\sum_j d_{ij} = 0 \ \ \forall i, \quad \sum_i d_{ij} = 0 \ \ \forall j.$$

Докажем более подробно единственность направления d. Обозначим d_B вектор компонент d, отвечающих базисным переменным.

Напомним, что в матрице A любая из $m+n$ строк линейно зависима от остальных, а остальные формируют линейно независимую систему строк. Поэтому имеем: $Ad = 0 \iff A'd = 0$, где A' есть матрица A после удаления из нее одной (любой) строки. Она имеет размер $(m+n-1) \times (mn)$. Из этой матрицы выберем столбцы, отвечающие базисным переменным. Получим матрицу A'_B размера $(m+n-1)\times(m+n-1)$. Как A'_N и A_N обозначим соответственно столбцы матриц A' и A, отвечающие вводимой в базис переменной (x_{kl}).

A'_B обратима. Действительно, если $\exists d_B \neq 0 : A'_B d_B = 0$, тогда выполнено и $A_B d_B = 0$, где A_B — матрица, полученная выбором столбцов, отвечающих базисным переменным, из матрицы A. Это верно, так как удаленная строка является линейной комбинацией оставшихся, для которых скалярное умножение на d_B дает 0, следовательно ее скалярное умножение на d_B также даст 0. Но столбцы A_B линейно независимы, стало быть ненулевой d_B не может быть решением $A'_B d_B = 0$. Так, $\ker A'_B = 0 \implies A'_B$ обратима. Более того, в силу линейной зависимости удаленной строки A от оставшихся,

$$A_B d_B = A_N \iff A'_B d_B = -A'_N \iff d_B = -(A'_B)^{-1} A'_N. \tag{11}$$

Отсюда, очевидно, d определяется единственным образом. Теперь построим его конструктивно, показав, что в матрице (d_{ij}) существует цикл.

Пусть множество номеров базисных переменных есть $B = \{i_1 j_1, \ldots, i_{m+n-1} j_{m+n-1}\}$. Также напомним, что вводимая в базис переменная имеет номер d_{kl}.

Для общности обозначим множество из номера строки k как

$$R_1 = \{k\}$$

На первом шаге выберем все базисные элементы из строки k. Они имеют номера

$$B_1 = \{ij \in B : i = k\} = \{ij \in B : i \in R_1\}.$$

Заметим, что это множество непусто, так как $\sum_j d_{kj} = 0$ и $d_{kl} = 1$, а из вида направления d помимо d_{kl} ненулевыми в строке k могут быть только соответствующие базисные элементы. Формально,

$$\sum_{\substack{j=1 \\ j\neq l}}^{n} d_{kj} + d_{kl} = 0 \implies \sum_{\substack{j=1 \\ j\neq l}}^{n} d_{kj} = \sum_{j:\, kj\in B} d_{kj} = -1 \implies \exists d_{kj},\ kj \in B : d_{kj} \neq 0.$$

На втором шаге рассмотрим столбцы, в которых стоят базисные элементы, выбранные на первом шаге. Причем на втором шаге выберем базисные элементы, отличные от уже выбранных ранее (то есть от элементов первой строки). Рассматриваемые столбцы имеют номера

$$C_1 = \{j : (\exists i : ij \in B_1)\} = \{j : kj \in B\}. \tag{12}$$

Выбираем базисные элементы, стоящие в столбцах с номерами, заданными (12), отличные от выбранных ранее. Номера этих элементов образуют множество

$$B_2 = \{ij \in (B \setminus B_1) : j \in C_1\}.$$

Эти элементы стоят в строках с номерами

$$R_2 = \{i : (\exists j : ij \in B_2)\}. \tag{13}$$

Множество B_2 непусто, так как

$$\begin{gathered} \sum_{j:\, kj \in B} d_{kj} = -1, \quad \sum_{\substack{i=1,\dots,m \\ j \in C_1}} d_{ij} = 0 \implies \\ \implies \sum_{\substack{i \in \{1,\dots,m\} \setminus R_1 \\ j \in C_1}} d_{ij} = 1 \implies \exists ij \in (\{1,\dots,m\} \setminus R_1) \times C_1 : d_{ij} \neq 0. \end{gathered} \tag{14}$$

Из (14) следует более наглядная форма записи множества номеров выбранных базисных элементов:

$$B_2 = \{ij \in B : i \in (\{1,\dots,m\} \setminus R_1),\ j \in C_1\} = \{ij \in B \cap \big((\{1,\dots,m\} \setminus R_1) \times C_1\big)\}.$$

Для наглядности опишем последующие 2 шага, после чего сформулируем общую схему.

На третьем шаге рассмотрим строки, в которых стоят базисные элементы, выбранные на втором шаге. Снова будем выбирать базисные элементы, отличные от выбранных ранее. Рассматриваемые строки имеют номера R_2. Выбираем базисные элементы, стоящие в этих строках, отличные от выбранных ранее. Номера этих элементов образуют множество

$$\begin{aligned} B_3 = \{ij \in (B \setminus B_2) : i \in R_2\} = \{ij : i \in R_2, j \in \{1,\dots,n\} \setminus C_1\} = \\ = \{ij \in B \cap \big(R_2 \times (\{1,\dots,n\} \setminus C_1)\big)\}. \end{aligned}$$

Эти элементы стоят в столбцах с номерами

$$C_2 = \{j : (\exists i : ij \in B_3)\} \tag{15}$$

Снова заметим тот факт, что полученное множество B_3 непусто. В столбцах с номерами C_1 базисные элементы, не стоящие в строках из R_1, находятся в строках из R_2. Отсюда имеем:

$$\begin{gathered}\sum_{\substack{i\in\{1,\dots,m\}\setminus R_1\\ j\in C_1}} d_{ij} = \sum_{\substack{i\in R_2\\ j\in C_1}} d_{ij} = 1, \qquad \sum_{\substack{i\in R_2\\ j\in\{1,\dots,n\}}} d_{ij} = 0 \implies \\ \implies \sum_{\substack{i\in R_2\\ j\in\{1,\dots,n\}\setminus C_1}} d_{ij} = -1 \implies \exists ij \in R_2\times(\{1,\dots,n\}\setminus C_1) : d_{ij} \neq 0.\end{gathered} \tag{16}$$

На четвертом шаге процедура аналогична второму шагу. Но здесь заметим, что алгоритм продолжается, если $l \notin C_2$, то есть если столбец, отвечающий вводимой в базис переменной, не был затронут на предыдущем шаге. Рассмотрим столбцы, в которых стоят базисные элементы, выбранные на третьем шаге. Выбираем базисные элементы этих столбцов, отличные от тех, что были выбраны ранее. Рассматриваемые столбцы имеют номера, задаваемые (15). Номера выбираемых на текущем шаге элементов образуют множество

$$B_4 = \{ij \in (B \setminus B_3) : j \in C_2\}.$$

Эти элементы стоят в строках с номерами

$$R_3 = \{i : (\exists j : ij \in B_3)\}. \tag{17}$$

Множество B_4 непусто. Действительно, в столбцах с номерами C_2 находятся все ненулевые (базисные) элементы строк с номерами R_2, отличные от тех, которые находятся в столбцах с номерами из C_1. Так,

$$\begin{gathered}\sum_{\substack{i\in R_2\\ j\in\{1,\dots,n\}\setminus C_1}} d_{ij} = \sum_{\substack{i\in R_2\\ j\in C_2}} d_{ij} = -1, \qquad \sum_{\substack{i\in\{1,\dots,m\}\\ j\in C_2}} d_{ij} = 0 \implies \\ \implies \sum_{\substack{i\in\{1,\dots,m\}\setminus R_2\\ j\in C_2}} d_{ij} = 1 \implies \exists ij \in (\{1,\dots,m\}\setminus R_2)\times C_2 : d_{ij} \neq 0.\end{gathered} \tag{18}$$

Из (18) следует более наглядная форма записи множества номеров выбранных базисных элементов:

$$B_4 = \{ij \in B \cap \big((\{1,\dots,m\}\setminus R_2)\times C_2\big)\} = \{ij \in B \cap \big(R_3 \times C_2\big)\}.$$

Далее будет рассмотрена общая форма очередного шага.

Пусть, как было сказано ранее, алгоритм завершается, если на нечетном шаге (при выборе новых базисных элементов в строках) встретится элемент с номером столбца l. То есть на шагах $2s-1, s \geq 2$ проверяется условие $l \in C_s$, при выполнении которого алгоритм завершается.

Докажем, что данный алгоритм обладает следующими свойствами:

1) ни один базисный элемент не встречается дважды, то есть $B_1 \cap B_2 \cap B_3 \cap \cdots = \emptyset$

2) алгоритм завершается за конечное число шагов

Для начала рассмотрим шаги с номерами $2s$ и $2s+1$, где $s \geq 1$. Пусть до текущего момента алгоритм не завершился (шаг с номером $2s-1$ не привел к завершению).

Пусть на шаге с номером $2s-1$ было получено множество столбцов C_s, а на шаге $2s-2$ было получено множество строк R_s. Пусть на шаге $2s-1$ получено условие

$$\sum_{\substack{i \in R_s \\ j \in C_s}} d_{ij} = -1 \tag{19}$$

Тогда на текущем $2s$-м шаге рассмотрим столбцы, в которых стоят базисные элементы, выбранные на $2s-1$-м шаге. Базисные элементы, выбранные на $2s-1$-м шаге стоят в столбцах с номерами C_s, в строках с номерами R_s. Выбираем базисные элементы из столбцов C_s, отличные от выбранных ранее (то есть стоящие в столбцах C_s и не в строках R_s).

Из (19) и условий на d (равенство нулю построчных и постолбцовых сумм элементов) следует, что можно выбрать новые базисные элементы согласно описанной процедуре, т.е. что существует по крайней мере один такой элемент:

$$\begin{gathered} \sum_{\substack{i \in R_s \\ j \in C_s}} d_{ij} = -1, \qquad \sum_{\substack{i \in \{1,\ldots,m\} \\ j \in C_s}} d_{ij} = 0 \implies \\ \implies \sum_{\substack{i \in \{1,\ldots,m\} \setminus R_s \\ j \in C_s}} d_{ij} = 1 \implies \exists ij \in (\{1,\ldots,m\} \setminus R_s) \times C_s : d_{ij} \neq 0. \end{gathered} \tag{20}$$

Пусть выбранные элементы стоят в строках с номерами R_{s+1}, то есть

$$R_{s+1} = \{i \in \{1,\ldots,m\} \setminus R_s : (\exists j \in C_s : ij \in B)\}.$$

Тогда множество элементов, выбираемых на текущем $2s$-м шаге запишем как

$$B_{2s} = \{ij \in B \cap (R_{s+1} \times C_s)\}.$$

Из (20) и того факта, что все базисные элементы, выбранные на данном шаге, стоят в строках с номерами R_{s+1}, имеем

$$\sum_{\substack{i\in\{1,\dots,m\}\setminus R_s\\ j\in C_s}} d_{ij} = \sum_{\substack{i\in R_{s+1}\\ j\in C_s}} d_{ij} = 1 \tag{21}$$

Далее переходим на следующий шаг (шаг с номером $2s+1$). Рассмотрим строки, в которых стоят базисные элементы, выбранные на $2s$-м шаге. Базисные элементы, выбранные на $2s$-м шаге стоят в столбцах с номерами C_s, в строках с номерами R_{s+1}. Выбираем базисные элементы из строк R_s, отличные от выбранных ранее (то есть стоящие в строках R_{s+1} и не в столбцах C_s).

Из (21) следует, что существует по крайней мере один такой элемент:

$$\begin{gathered}\sum_{\substack{i\in R_{s+1}\\ j\in C_s}} d_{ij} = 1, \qquad \sum_{\substack{i\in R_{s+1}\\ j\in\{1,\dots,n\}}} d_{ij} = 0 \implies \\ \implies \sum_{\substack{i\in R_{s+1}\\ j\in\{1,\dots,n\}\setminus C_s}} d_{ij} = -1 \implies \exists ij \in R_{s+1}\times(\{1,\dots,n\}\setminus C_s) : d_{ij}\neq 0.\end{gathered} \tag{22}$$

Пусть выбранные элементы стоят в столбцах с номерами C_{s+1}, то есть

$$C_{s+1} = \{j\in\{1,\dots,n\}\setminus C_s : (\exists i\in R_{s+1} : ij\in B)\}.$$

Множество выбираемых на данном $(2s+1)$-м шаге базисных элементов запишем как

$$B_{2s+1} = \{ij\in B\cap(R_{s+1}\times C_{s+1})\}$$

Если $l\in C_{s+1}$, завершаем алгоритм.

Из (22) и того факта, что все базисные элементы, выбранные на данном шаге, стоят в столбцах с номерами C_{s+1}, имеем

$$\sum_{\substack{i\in R_{s+1}\\ j\in\{1,\dots,n\}\setminus C_s}} d_{ij} = \sum_{\substack{i\in R_{s+1}\\ j\in C_{s+1}}} d_{ij} = -1 \tag{23}$$

Из (19)–(23) видно, что в рассматриваемых на четных шагах строках и рассматриваемых на нечетных шагах столбцах базисные элементы, отличные от выбранных на соответственно предыдущих шагах, всегда существуют.

В итоге, на каждом шаге алгоритм проходит через хотя бы один базисный элемент. Отсюда, если базисные элементы не повторяются, алгоритм после не более чем $m+n-1$ шагов обязан завершиться. Действительно,

$$\sum_i d_{il} = 0, \quad d_{kl} = 1 \implies \sum_{\substack{i \in \{1,\dots,m\} \\ i \neq k}} d_{il} = -1 \implies \exists ij \in B : j = l,$$

то есть в столбце l встречается хотя бы 1 базисный элемент. Алгоритм может пройти элемент из столбца l только на нечетном шаге, так как иначе, если элемент из столбца l получен на четном шаге, это бы означало, что алгоритм выбрал столбец с номером l на предыдущем, нечетном, шаге. Напомним, что это верно, так как на четных шагах рассматриваются столбцы, выбранные на нечетных. Но это бы значило, что алгоритм должен был бы завершиться на предыдущем шаге. Противоречие. И отсюда получаем, что при отсутствии повторов вершин алгоритм на некотором шаге все же пройдет базисный элемент, стоящий в столбце l, а следовательно завершится.

Покажем, что базисные элементы повторяться не могут (алгоритм завершается на некотором шаге q и $B_1 \cap B_2 \cap \dots \cap B_q = \emptyset$). Пусть на некотором шаге p алгоритма в построенном множестве B_p существует элемент из одного из множеств, построенных на более ранних шагах. Пусть этот элемент встречался во множестве B_r, построенном на шаге r. Очевидно, повторение может произойти не раньше, чем через 3 шага. Действительно, пусть данный шаг нечетный (четный). Тогда на предыдущем шаге рассматривались элементы из столбцов (строк), отличных от столбцов (строк), в которых стоят элементы на текущем шаге. И на шаге, предшествующем предыдущему, рассматривались элементы тех же столбцов (строк), что и элементы на предыдущем шаге, значит все элементы двух предыдущих шагов были в других столбцах (строках), нежели элементы текущего. Так, $(r \leq p-3)$.

Рассмотрим первое повторение какого-либо элемента.

Пусть повторение произошло на нечетном шаге $2s-1$. Это значит, что элемент $ij_1 \in B_r$ входит в ту же строку i, что и некоторый элемент $ij_2 \in B_{2s-1}$. Для начала пусть r был четным ($r = 2k$). Это значит, что на шаге $r+1$ рассматривалась в том числе строка i. И есть два варианта: либо $ij_1, ij_2 \in B_r$, либо $ij_1 \in B_r, ij_2 \in B_{r+1}$. В обоих случаях получаем противоречие, так как повторяющийся элемент должен встречаться на 3 или более шагов ранее, а ij_2 не является повторяющимся (первое повторение, по

предположению, произошло на шаге $2s-1$, а ij_2 встретился на шаге $2s-2$). Аналогично, если r нечетный, то на шаге B_r рассматривалась i-я строка, а значит либо $ij_1, ij_2 \in B_r$, либо $ij_1 \in B_r, ij_2 \in B_{r-1}$. Противоречие в обоих случаях. Так, повторение не могло произойти на нечетном шаге.

Совершенно аналогично если первое повторение произошло на четном шаге $2s$, это значит, что $i_1j \in B_r$ входит в тот же столбец j, что и некоторый элемент $i_2j \in B_{2s}$. Тогда элемент i_2j не мог встретиться на 3 или более шагов позже, чем i_1j.

Получаем противоречие. Значит, повторений происходить не может, и алгоритм действительно завершается, причем за не более чем $m+n-1$ шаг (т.к. $|B| = m+n-1$), попав в базисный элемент в столбце l на некотором нечетном шаге.

Пусть алгоритм завершился за p шагов. Пусть элемент, полученный на последнем шаге алгоритма, есть $i_pj_p = i_pl$. Тогда на предыдущем шаге $(p-1)$ в строке i_p находился некоторый элемент $i_{p-1}j_{p-1} = i_pj_{p-1} \in B_{p-1}$. Это значит, что на шаге $p-2$ в столбце j_{p-1} стоял некоторый элемент $i_{p-2}j_{p-2} = i_{p-2}j_{p-1} \in B_{p-2}$. Рассуждения повторяются до первых шагов. В B_2 таким образом выделяется элемент i_2j_2. На шаге 2 в столбце j_2 стоял некоторый элемент $i_2j_2 = i_1j_2 \in B_1$, то есть некоторый элемент первой строки i_1j_1. Получили набор номеров базисных переменных $I = \{i_1j_1, \ldots, i_pj_p\} \subset B$. Предположим,

$$d_{ij} = \begin{cases} 1, & ij = kl \text{ или } ij = i_sj_s,\ s \text{ четное} \\ -1, & ij = i_sj_s,\ s \text{ нечетное} \\ 0, & \text{иначе} \end{cases}$$

Покажем выполнение $Ad = 0$. Возьмем любую строку i. Либо в данной строке нет элементов из $I \cup \{kl\}$, тогда ограничение $\sum_j d_{ij} = 0$ для данной строки выполнено. Либо в этой строке есть элемент из этого множества, то есть $ij : d_{ij} \neq 0$. Заметим, что во множестве $I \cup \{kl\}$ четное число элементов, а в силу чередования -1 и 1 при присваивании $d_{i_sj_s}$ значений количество элементов вектора d, равных 1, равно количеству элементов этого вектора, равных -1. Рассмотрим строки, в которых стоят элементы $ij : d_{ij} = -1$. Возьмем любую такую строку, например, с номером i. -1 присваивается элементам d с номерами $i_sj_s \in I$, где s нечетное. Пусть как раз такой элемент i_sj_s находится в данной строке. По построению множества I в той же строке i есть элемент $i_{s-1}j_{s-1} \in B_{s-1}$, для которого соответствующему $d_{i_{s-1}j_{s-1}}$ присвоена 1 (где для $s = 1$ положим $i_0j_0 = kl$, $B_0 = \{kl\}$).

При этом базисные элементы, стоящие в данной строке i, принадлежат либо B_{s-1}, либо B_s. Итого, для данной строки ровно для одного элемента $i_s j_s$ из B_s соотвествующему $d_{i_s j_s}$ присвоена -1 и ровно для одного элемента $i_{s-1} j_{s-1}$ из $\in B_{s-1}$ присвоена 1, и других элементов, соответствующих ненулевым d_{ij}, в этой строке нет. Так, выполнены первые m ограничений в $Ad = 0$, ведь кроме строк с элементами -1 ненулевых строк нет (все элементы 1 стоят ровно в тех же строках, ведь на каждую строку с -1 приходится элемент 1, и их одинаковое количество).

Аналогично можно показать выполнение ограничений по столбцам. Возьмем любой столбец j. Либо в данном столбце нет элементов из $I \cup \{kl\}$, тогда ограничение $\sum_i d_{ij} = 0$ для данного столбца выполнено. Либо в этом столбце есть элемент из этого множества, то есть $ij : d_{ij} \neq 0$. Рассмотрим столбцы, в которых стоят элементы $ij : d_{ij} = -1$. Возьмем любой такой столбец, например, с номером j. Пусть элемент $i_s j_s$ с нечетным s находится в данном столбце. По построению множества I в том же столбце j есть элемент $i_{s+1} j_{s+1} \in B_{s+1}$, для которого соответствующему $d_{i_{s+1} j_{s+1}}$ присвоена 1 (где для $s = p$ положим $i_{p+1} j_{p+1} = kl$, $B_{p+1} = \{kl\}$). При этом базисные элементы, стоящие в данном столбце j, принадлежат либо B_s, либо B_{s+1}. Итого, для данного столбца ровно для одного элемента $i_s j_s$ из B_s соотвествующему $d_{i_s j_s}$ присвоена -1 и ровно для одного элемента $i_{s+1} j_{s+1}$ из $\in B_{s+1}$ присвоена 1, и других элементов, соответствующих ненулевым d_{ij}, в этом столбце нет. Так, выполнены ограничения $m+1, \ldots, m+n$ в $Ad = 0$, так как рассмотрены столбцы с элементами -1, и в каждом таком столбце есть элемент 1, а так как количество -1 и 1 равно, то других ненулевых столбцов нет.

Итак, был полностью обоснован алгоритм выбора направления d, удовлетворяющего $Ad = 0$ и свойствам, указанным в формулировке данного следствия. Следствие доказано.

$\square$

3 Практические аспекты

3.1 Замечания к алгоритму

Машинным вычислениям свойственна некоторая погрешность в силу специфичности машинного представления чисел. С другой стороны, ограничения задачи (1)–(4) могут нарушаться даже при совсем небольших ошибках вычислений.

При целочисленных ограничениях решение можно найти в целых числах (каждая вершина целочисленная). При целочисленных коэффициентах в целевой функции пересчет потенциалов также возможен в целых числах. В общем же случае используются параметры точности.

3.2 Программная реализация

В рамках работы была написана программная реализация метода решения транспортной задачи с использованием языка программирования Python и стандартных библиотек. Исходный код проекта, включая результаты экспериментов, доступен в GitHub-репозитории[1]. Листинг основной программы приведен в приложении А.1.

[1] `https://github.com/oscar-foxtrot/prac-io-0/tree/main/year_4_assignment_2` (дата обращения: 21.03.2026)

4 Эксперименты

Пример 1.

$$f = 2x_{11} + 3x_{12} + x_{13} + 5x_{21} + 4x_{22} + 8x_{23} + 5x_{31} + 6x_{32} + 8x_{33} \to \min$$

$$x_{11} + x_{12} + x_{13} = 20, \quad x_{21} + x_{22} + x_{23} = 30, \quad x_{31} + x_{32} + x_{33} = 25,$$

$$x_{11} + x_{21} + x_{31} = 10, \quad x_{12} + x_{22} + x_{32} = 35, \quad x_{13} + x_{23} + x_{33} = 30,$$

$$x_{ij} \geq 0$$

Найденное решение:

$$X = \begin{bmatrix} 0 & 0 & 20 \\ 0 & 30 & 0 \\ 10 & 5 & 10 \end{bmatrix}, \quad \text{Суммарные затраты: } 300$$

Пример 2.

$$f = 5x_{11} + 3x_{12} + x_{13} + 3x_{21} + 2x_{22} + 4x_{23} + 4x_{31} + x_{32} + 2x_{33} \to \min$$

$$x_{11} + x_{12} + x_{13} = 10, \quad x_{21} + x_{22} + x_{23} = 20, \quad x_{31} + x_{32} + x_{33} = 30,$$

$$x_{11} + x_{21} + x_{31} = 15, \quad x_{12} + x_{22} + x_{32} = 20, \quad x_{13} + x_{23} + x_{33} = 25,$$

$$x_{ij} \geq 0$$

Найденное решение:

$$X = \begin{bmatrix} 0 & 0 & 10 \\ 15 & 5 & 0 \\ 0 & 15 & 15 \end{bmatrix}, \quad \text{Суммарные затраты: } 110$$

Пример 3.

$$f = 4x_{11} + 8x_{12} + 5x_{13} + 6x_{21} + 7x_{22} + 3x_{23} \to \min$$

$$x_{11} + x_{12} + x_{13} = 30, \quad x_{21} + x_{22} + x_{23} = 50,$$

$$x_{11} + x_{21} = 20, \qquad x_{12} + x_{22} = 40, \quad x_{13} + x_{23} = 20,$$

$$x_{ij} \geq 0$$

Найденное решение:

$$X = \begin{bmatrix} 20 & 10 & 0 \\ 0 & 30 & 20 \end{bmatrix}, \quad \text{Суммарные затраты: } 430$$

Пример 4.

$$f = 3x_{11} + x_{12} + 7x_{13} + 4x_{14} + 2x_{21} + 6x_{22} + 5x_{23} + 9x_{24} +$$

$$8x_{31} + 3x_{32} + 3x_{33} + 2x_{34} + 4x_{41} + 7x_{42} + 2x_{43} + 5x_{44} \to \min$$

$$\sum_{j=1}^{4} x_{1j} = 20, \quad \sum_{j=1}^{4} x_{2j} = 30, \quad \sum_{j=1}^{4} x_{3j} = 25, \quad \sum_{j=1}^{4} x_{4j} = 15,$$

$$\sum_{i=1}^{4} x_{i1} = 15, \quad \sum_{i=1}^{4} x_{i2} = 25, \quad \sum_{i=1}^{4} x_{i3} = 30, \quad \sum_{i=1}^{4} x_{i4} = 20,$$

$$x_{ij} \geq 0$$

Найденное решение:

$$X = \begin{bmatrix} 0 & 20 & 0 & 0 \\ 15 & 0 & 15 & 0 \\ 0 & 5 & 0 & 20 \\ 0 & 0 & 15 & 0 \end{bmatrix}, \quad \text{Суммарные затраты: } 210$$

Пример 5.

$$f = \sum_{i=1}^{6} \sum_{j=1}^{5} c_{ij} x_{ij} \to \min$$

$$\sum_{j=1}^{5} x_{1j} = 40, \quad \sum_{j=1}^{5} x_{2j} = 50, \quad \sum_{j=1}^{5} x_{3j} = 30,$$

$$\sum_{j=1}^{5} x_{4j} = 60, \quad \sum_{j=1}^{5} x_{5j} = 35, \quad \sum_{j=1}^{5} x_{6j} = 35,$$

$$\sum_{i=1}^{6} x_{i1} = 30, \quad \sum_{i=1}^{6} x_{i2} = 45, \quad \sum_{i=1}^{6} x_{i3} = 60,$$

$$\sum_{i=1}^{6} x_{i4} = 50, \quad \sum_{i=1}^{6} x_{i5} = 65,$$

$$x_{ij} \geq 0$$

Найденное решение:

$$X_1 = \begin{bmatrix} 0 & 10 & 0 & 15 & 15 \\ 0 & 0 & 0 & 0 & 50 \\ 0 & 0 & 0 & 30 & 0 \\ 30 & 0 & 30 & 0 & 0 \\ 0 & 0 & 30 & 5 & 0 \\ 0 & 35 & 0 & 0 & 0 \end{bmatrix}, \quad \text{Суммарные затраты: } 1690$$

Заключение

В ходе работы был успешно реализован и протестирован метод решения транспортной задачи. Эксперименты подтвердили, что метод находит оптимальное решение за конечное число шагов на различных примерах.

Метод является более эффективным, нежели симплекс-метод (меньше размер таблицы), в силу специальной структуры задачи, в чем состоит одно из основных его преимуществ.

A Приложение

A.1 Программная реализация метода с использованием ЯП Python

```python
import numpy as np

def find_nwc(supply, demand, tol=1e-12):
    """
    Нахождение начального базисного допустимого решения
    методом северо-западного угла
    input:
    supply, demand --- векторы ограничений на
    спрос и предложение соответственно
    tol --- параметр точности
    output:
    x --- начальное базисное допустимое решение,
    представленное в виде матрицы размера m x n
    basics --- номера базисных переменных
    """
    m, n = len(supply), len(demand)
    x = np.zeros((m, n))
    i = j = 0

    supply = supply.copy()
    demand = demand.copy()

    basics = set()

    while i < m and j < n:

        # частичная защита от ошибок округления floating-point чисел
        if abs(supply[i]) < tol:
            supply[i] = 0
```

```
        if abs(demand[j]) < tol:
            demand[j] = 0

        basics.add((i, j))
        x[i, j] = min(supply[i], demand[j])

        if supply[i] < demand[j]:
            demand[j] -= x[i, j]
            i += 1
        else:
            supply[i] -= x[i, j]
            if j < n - 1:
                j += 1
            else:  # случай, если есть нулевые ограничения
                i += 1

    return x, basics


def find_potentials(x, c, basics):
    """
    Вычисление потенциалов
    input:
    x --- матрица поставок размера m x n
    c --- матрица, задающая целевую функцию (m x n)
    basics --- множество номеров базисных переменных
    output:
    u, v --- векторы потенциалов
    (u --- размера m, v --- размера n)
    """
    m, n = x.shape
    u = np.full(m, np.nan)
    v = np.full(n, np.nan)
```

```python

    # один потенциал фиксируется (принимается равным 0)
    u[0] = 0
    updated = True
    while updated:
        updated = False
        for i in range(m):
            for j in range(n):
                # для базисных переменных по мере возможности
                # решаются уравнения на потенциалы
                # (метод распространения)
                if (i, j) in basics:
                    if not np.isnan(u[i]) and np.isnan(v[j]):
                        v[j] = c[i, j] - u[i]
                        updated = True
                    elif np.isnan(u[i]) and not np.isnan(v[j]):
                        u[i] = c[i, j] - v[j]
                        updated = True

    return u, v


def find_reduced_costs(x, c, u, v, tol=1e-12):
    """
    Вычисление значений симплекс-множителей (\\delta_{ij})
    input:
    x --- матрица поставок размера m x n
    c --- матрица, задающая целевую функцию (m x n)
    u, v --- потенциалы
    tol --- параметр точности
    output:
    delta --- матрица симплекс-множителей размера m x n
    """
```

```
    m, n = x.shape
    delta = np.zeros((m, n))

    for i in range(m):
        for j in range(n):
            reduced_cost = c[i, j] - (u[i] + v[j])
            if abs(reduced_cost) < tol:
                reduced_cost = 0
            delta[i, j] = reduced_cost

    return delta


def find_entering(delta, x):
    """
    Нахождение вводимой в базис переменной
    с минимальным индексом
    input:
    x --- матрица поставок размера m x n
    delta --- матрица симплекс-множителей размера m x n
    output:
    (i, j) или None --- номер вводимой переменной
    """

    # найти переменную, которой соответствует
    # отрицательный симплекс-множитель
    for i in range(delta.shape[0]):
        for j in range(delta.shape[1]):
            if delta[i, j] < 0:
                return (i, j)


def find_d_cyclic(basics, kl_entering, m, n):
```

```
    """
    Построение допустимого направления невозрастания функции
    путем нахождения цикла через базисные переменные и
    вводимую переменную с использованием матричного метода,
    представленного в описании алгоритма
    (github.com/oscar-foxtrot/prac-io-0/tree/main/year_4_assignment_2)
    input:
    basics --- множество номеров базисных переменных
    kl_entering --- номер вводимой переменной
    n, m --- числа, задающие размерность
    output:
    d --- допустимое направление невозрастания целевой функции
    """

    visited_sets = list()

    R_prev = {kl_entering[0]}
    C_prev = {kl_entering[1]}
    basics_visited = set()
    R = {i for i in range(m)}
    C = {i for i in range(n)}

    new_basics = set()
    while True:

        # строковый шаг (нечетный)
        # выбрать непосещенные базисные переменные из текущей строки
        new_basics = {(i, j) for i in R_prev for j in \
            C - C_prev if (i, j) in basics}
        visited_sets += [new_basics]

        if any(j == kl_entering[1] for (i, j) in new_basics):
            # достигли столбца вводимой переменной => закончить шаги
```

```
            break

        # обновление множества текущих столбцов
        C_prev = {j for (i, j) in new_basics}

        # столбцовый шаг (четный)
        # выбрать непосещенные базисные переменные из текущего столбца
        new_basics = {(i, j) for j in C_prev for i in \
            R - R_prev if (i, j) in basics}
        visited_sets += [new_basics]

        # обновление множества текущих строк
        R_prev = {i for (i, j) in new_basics}

    # нахождение цикла (обратный ход):
    current_set_number_from_the_end = 1
    current_basic_var = kl_entering
    cycle = []
    while True:
        # переходим к следующей базисной позиции по столбцу
        for i in range(m):
            if (i, current_basic_var[1]) in \
                visited_sets[-current_set_number_from_the_end]:
                current_basic_var = (i, current_basic_var[1])
                cycle += [current_basic_var]
                break
        current_set_number_from_the_end += 1

        # переходим к следующей базисной позиции по строке
        # завершаем обратный ход, если достигнута
        # строка вводимой переменной
        if current_basic_var[0] == kl_entering[0]:
            break
```

```
        for j in range(n):
            if (current_basic_var[0], j) in \
                visited_sets[-current_set_number_from_the_end]:
                current_basic_var = (current_basic_var[0], j)
                cycle += [current_basic_var]
                break
        current_set_number_from_the_end += 1

    # присваивание d значений 1, -1 по циклу
    d = np.zeros((m, n))
    d[kl_entering] = 1
    sign = -1
    for (i, j) in cycle:
        d[(i,j)] = sign
        sign = -sign

    return d


def pivot(x, d, tol=1e-12):
    """
    Найти новое базисное допустимое решение вдоль направления d,
    найти выводимую из базиса переменную с минимальным индексом
    input:
    x --- матрица поставок размера m x n
    d --- допустимое направление невозрастания целевой функции
    tol --- параметр точности
    output:
    x_star --- матрица поставок размера m x n, задающая новое
    базисное допустимое решение
    pq_leaving --- номер выводимой из базиса переменной
    """

```

```
    # максимальный шаг находится из условия theta = np.min(x[d < 0])
    # где d < 0 <=> d == -1

    m = x.shape[0]
    n = x.shape[1]
    theta = None
    for i in range(m):
        for j in range(n):
            if d[i, j] < 0 \
                and (theta is None or x[i, j] < theta):
                pq_leaving = (i, j)
                theta = x[i, j]

    x_star = x + theta * d

    # частичная защита от ошибок округления floating-point чисел
    x_star[np.abs(x_star) < tol] = 0

    return x_star, pq_leaving


def solve_transportation_problem(cost, supply, demand, tol=1e-12):
    """
    Найти решение транспортной задачи
    input:
    supply, demand --- векторы ограничений на
    спрос и предложение соответственно
    cost --- матрица, задающая целевую функцию (m x n)
    tol --- параметр точности
    output:
    x --- матрица поставок размера m x n, задающая
    решение транспортной задачи
    """
```

```python

    # найти начальное базисное допустимое решение
    # методом северо-западного угла
    x, basics = find_nwc(supply, demand, tol)

    while True:
        # вычислить потенциалы и симплекс-множители
        u, v = find_potentials(x, cost, basics)
        delta = find_reduced_costs(x, cost, u, v, tol)

        # найти вводимую в базис переменную или завершить шаги,
        # если найденное решение оптимальное
        entering = find_entering(delta, x)
        if entering is None:
            break

        # обновить x, обновить номера базисных переменных
        d = find_d_cyclic(basics, entering, x.shape[0], x.shape[1])
        x, leaving = pivot(x, d, tol)
        basics.add(entering)
        basics -= {leaving}

    return x
```

Список литературы

1. *Bland R. G.* New Finite Pivoting Rules for the Simplex Method // Mathematics of Operations Research. — 1977. — Vol. 2. — P. 103–107.
2. *Dantzig G. B.* Linear Programming and Extensions. — Santa Monica, CA : RAND Corporation, 1963.
3. *Hitchcock F. L.* The Distribution of a Product from Several Sources to Numerous Localities // Journal of Mathematics and Physics. — 1941. — Vol. 20. — P. 224–230.
4. *Korte B.*, *Vygen J.* Combinatorial Optimization: Theory and Algorithms. — 5th edition. — Springer Berlin, Heidelberg, 2012. — P. 51–70.
5. *Rao S.* Engineering Optimization: Theory and Practice. — 4th edition. — John Wiley & Sons, Inc., 2009. — P. 119–248.